

Noora Health – Data Engineer Assignment

This document describes the end-to-end ELT pipeline built to ingest, transform, validate, and analyze WhatsApp intervention data. The raw data consists of two append only tables (messages and statuses). The solution preserves full lifecycle history while enabling analytical use cases.

1. Data Ingestion (Airbyte → BigQuery)

```
GitHub (messages.csv, statuses.csv)
  ↓
Airbyte Cloud (HTTPS Source)
  ↓
BigQuery Dataset: noora_health_raw
  ↓
Tables:
- noora_raw_files (messages)
- noora_statuses_file (statuses)
```

The raw CSV files were hosted on GitHub and ingested into BigQuery using Airbyte Cloud. Since both source tables are append-only, no transformation was performed at ingestion time. The raw layer remains immutable and serves as the source of truth.

2. Transformation Logic

Deduplicate Messages (Append-Only Handling)

```
WITH latest_messages AS (
  SELECT *
  FROM (
    SELECT *,
      PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', inserted_at) AS
inserted_at_ts,
      ROW_NUMBER() OVER (
```

```

        PARTITION BY id
        ORDER BY PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', inserted_at)
DESC
    ) AS rn
FROM `noora_health_raw.noora_raw_files`
)
WHERE rn = 1
)

```

The messages table is append-only. If a message changes, a new row is inserted instead of updating the existing record.

To produce one row per logical message, the latest version was selected using ROW_NUMBER() ordered by inserted_at.

This ensures:

- Only one record per message_id
- No loss of final message state

Preserve Full Status Lifecycle

```

status_history AS (
  SELECT
    message_id,
    ARRAY_AGG(
      STRUCT(
        status,
        PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', timestamp) AS timestamp
      )
      ORDER BY PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', timestamp)
    ) AS status_events
  FROM `noora_health_raw.noora_statuses_file`
  GROUP BY message_id
)

```

The statuses table is also append-only and records lifecycle transitions (sent, delivered, read, failed).

Instead of flattening or selecting only the latest status, all status events were aggregated into an ordered ARRAY.

This preserves:

- Complete message lifecycle
- Temporal ordering
- Historical traceability

Final Analytical Table

```
CREATE OR REPLACE TABLE noora_health_analytics.messages_enriched AS

WITH latest_messages AS (
  SELECT *
  FROM (
    SELECT *,
           PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', inserted_at) AS
inserted_at_ts,
           ROW_NUMBER() OVER (
             PARTITION BY id
             ORDER BY PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', inserted_at)
DESC
           ) AS rn
    FROM `noora_health_raw.noora_raw_files`
  )
  WHERE rn = 1
),

status_history AS (
  SELECT
    message_id,
    ARRAY_AGG(
      STRUCT(
        status,
        PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', timestamp) AS timestamp
      )
      ORDER BY PARSE_TIMESTAMP('%m/%d/%Y %H:%M:%S', timestamp)
    ) AS status_events
  FROM `noora_health_raw.noora_statuses_file`
  GROUP BY message_id
)

SELECT
  m.id AS message_id,
  m.masked_author,
  m.masked_addressees,
  m.content,
  m.direction,
  m.inserted_at_ts AS message_inserted_at,
  s.status_events
FROM latest_messages m
```

```
LEFT JOIN status_history s
ON m.id = s.message_id;
```

The final table contains one row per logical message.

Design characteristics:

- Deduplicated message records
- Full lifecycle preserved
- User identifiers included for analytics
- Proper TIMESTAMP typing for downstream calculations

3. Data Validation

Duplicate Message Detection

```
WITH duplicates AS (
  SELECT
    m1.message_id AS msg1,
    m2.message_id AS msg2,
    m1.content,
    m1.message_inserted_at AS inserted_at_1,
    m2.message_inserted_at AS inserted_at_2
  FROM `noorahealth-488108.noora_health_analytics.messages_enriched` m1
  JOIN `noorahealth-488108.noora_health_analytics.messages_enriched` m2
    ON m1.content = m2.content
    AND m1.message_id < m2.message_id
    AND ABS(
      TIMESTAMP_DIFF(
        m1.message_inserted_at,
        m2.message_inserted_at,
        SECOND
      )
    ) <= 60
)

SELECT *
FROM duplicates;
```

Explanation

Messages were considered potential duplicates if:

- Content is identical
- Inserted within 60 seconds
- Message IDs are different

This flags suspicious retries while avoiding self-matching rows.

Referential Integrity Check

```
SELECT s.message_id
FROM `noorahealth-488108.noora_health_raw.noora_statuses_file` s
LEFT JOIN `noorahealth-488108.noora_health_raw.noora_raw_files` m
  ON s.message_id = m.id
WHERE m.id IS NULL;
```

Explanation

This query checks for orphan status records.

Result: No rows returned, confirming that all status events correspond to a valid message.

Lifecycle Temporal Integrity

```
WITH flattened AS (
  SELECT
    message_id,
    s.status,
    s.timestamp
  FROM `noorahealth-488108.noora_health_analytics.messages_enriched`,
  UNNEST(status_events) s
),

sent_times AS (
  SELECT
    message_id,
    MIN(timestamp) AS sent_time
  FROM flattened
  WHERE status = 'sent'
  GROUP BY message_id
)
```

```
SELECT f.*
FROM flattened f
JOIN sent_times s
  ON f.message_id = s.message_id
WHERE f.status = 'read'
  AND f.timestamp < s.sent_time;
```

Explanation

This validation ensures lifecycle ordering:

sent → delivered → read

The query checks whether any "read" timestamp occurs before the earliest "sent" timestamp.

Result: No temporal inconsistencies detected.

Outbound Messages Without Status Events

```
SELECT *
FROM `noorahealth-488108.noora_health_analytics.messages_enriched`
WHERE direction = 'outbound'
  AND (status_events IS NULL OR ARRAY_LENGTH(status_events) = 0);
```

Outbound messages should typically have at least one lifecycle status (sent, delivered, read, or failed).

This query identifies outbound messages without any recorded status events.

4. Data Visualization

Dependencies

```
pip install google-cloud-bigquery pandas plotly
```

```
pip install db-dtypes
```

```
import os
```

```
from google.cloud import bigquery
```

Setting Up Environment

```
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = r"C:/Users/your_json_key"
```

```
client = bigquery.Client()
```

Total Users vs Active Users Over Time

Definitions:

Total users = distinct users who sent OR received a message

Active users = distinct users who sent inbound messages

```
query_users = """
WITH base AS (
    SELECT
        DATE_TRUNC(DATE(message_inserted_at), WEEK) AS week,
        CASE
            WHEN direction = 'inbound' THEN masked_author
            WHEN direction = 'outbound' THEN masked_addressees
        END AS user_id,
        direction
    FROM `noorahealth-488108.noora_health_analytics.messages_enriched`
)

SELECT
    week,
    COUNT(DISTINCT user_id) AS total_users,
    COUNT(DISTINCT IF(direction = 'inbound', user_id, NULL)) AS active_users
FROM base
GROUP BY week
ORDER BY week
"""
```

```
df_users = client.query(query_users).to_dataframe()
df_users.head()
```

```
import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=df_users["week"],
    y=df_users["total_users"],
    mode="lines+markers",
    name="Total Users"
))

fig.add_trace(go.Scatter(
    x=df_users["week"],
    y=df_users["active_users"],
    mode="lines+markers",
    name="Active Users"
))

fig.update_layout(
    title="Total vs Active Users Over Time",
    xaxis_title="Week",
    yaxis_title="Number of Users",
    template="plotly_white"
)

fig.show()
```

Fraction of Non-Failed Outbound Messages That Were Read

Definition:

Considered outbound messages only

Excluded messages with status "failed"

```
query_read_rate = """
WITH flattened AS (
    SELECT
        message_id,
```



```

        direction,
        s.status
    FROM `noorahealth-488108.noora_health_analytics.messages_enriched`,
    UNNEST(status_events) s
    WHERE direction = 'outbound'
)

SELECT
    COUNT(DISTINCT IF(status != 'failed', message_id, NULL)) AS non_failed,
    COUNT(DISTINCT IF(status = 'read', message_id, NULL)) AS read_messages
FROM flattened
"""

```

```
df_read = client.query(query_read_rate).to_dataframe()
```

```
read_rate = df_read["read_messages"][0] / df_read["non_failed"][0]
read_rate
```

```
print(f"Read Rate: {read_rate:.2%}")
```

```
import plotly.graph_objects as go
```

```

labels = ["Read", "Not Read"]
values = [df_read["read_messages"][0], df_read["non_failed"][0] -
df_read["read_messages"][0]]

```

```

fig = go.Figure(go.Pie(
    labels=labels,
    values=values,
    hole=0.6,
    marker_colors=["#2ecc71", "#e0e0e0"]
))

```

```

fig.update_layout(
    title="Outbound Message Read Rate",
    annotations=[dict(text=f"{values[0]/sum(values):.1%}", font_size=24,
showarrow=False)]
)

```

```
fig.show()
```

Distribution of Time Between Sent and Read

read_timestamp - sent_timestamp

```
query_read_time = """
WITH flattened AS (
    SELECT
        message_id,
        s.status,
        s.timestamp
    FROM `noorahealth-488108.noora_health_analytics.messages_enriched`,
    UNNEST(status_events) s
    WHERE direction = 'outbound'
),

sent_times AS (
    SELECT
        message_id,
        MIN(timestamp) AS sent_time
    FROM flattened
    WHERE status = 'sent'
    GROUP BY message_id
),

read_times AS (
    SELECT
        message_id,
        MIN(timestamp) AS read_time
    FROM flattened
    WHERE status = 'read'
    GROUP BY message_id
)

SELECT
    TIMESTAMP_DIFF(r.read_time, s.sent_time, SECOND) AS seconds_to_read
FROM sent_times s
JOIN read_times r
    ON s.message_id = r.message_id
WHERE r.read_time IS NOT NULL
"""

df_read_time = client.query(query_read_time).to_dataframe()
```

```
df_read_time.head()
```

```
import plotly.express as px
```

```
fig = px.histogram(  
    df_read_time,  
    x="seconds_to_read",  
    nbins=50,  
    title="Distribution of Time to Read (Seconds)"  
)
```

```
fig.show()
```

Outbound Messages in the Last Week by Status

```
query_last_week = """  
WITH flattened AS (  
    SELECT  
        DATE(s.timestamp) AS status_date,  
        s.status  
    FROM `noorahealth-488108.noora_health_analytics.messages_enriched`,  
    UNNEST(status_events) s  
    WHERE direction = 'outbound'  
)  
  
max_date AS (  
    SELECT MAX(status_date) AS max_status_date  
    FROM flattened  
)  
  
SELECT  
    f.status,  
    COUNT(*) AS message_count  
FROM flattened f  
CROSS JOIN max_date m  
WHERE f.status_date BETWEEN DATE_SUB(m.max_status_date, INTERVAL 6 DAY)  
        AND m.max_status_date  
  
GROUP BY f.status  
ORDER BY message_count DESC  
"""
```

```
df_last_week = client.query(query_last_week).to_dataframe()
df_last_week
```

```
import plotly.express as px
```

```
fig = px.bar(
    df_last_week,
    x="status",
    y="message_count",
    title="Outbound Messages (Last 7 Days in Dataset) by Status",
    template="plotly_white"
)

fig.show()
```